

Universidade Federal de Viçosa – Campus Florestal
Bacharelado em Ciência da Computação

SISTEMAS DISTRIBUÍDOS

PROFESSOR(A): Thais Regina de M. B. Silva

SISTEMAS DISTRIBUÍDOS

DEPLOY WITH MIDDLEWARE:

Gabriel Santos Ferreira de Pádua - 4705

Álvaro Gomes da Silva Neto - 5095

Lucas Borges Moura e Silva - 4689



Introdução - Garden in Overworld.....	3
Organização:.....	4
Cliente:.....	5
Servidor:.....	5
Componentes (Servidor):.....	6
Camada Controllers.....	6
Camada Models.....	7
Camada Views.....	8
mainServer.py.....	8
Componentes (Cliente).....	9
Camada Controller.....	9
Camada Models.....	10
Camada View.....	11
mainClient.py.....	11
Middleware VS. Sockets.....	12
Alterações:.....	12
Dificuldades:.....	13
Uso das Threads (Refatoração para Middleware RMI).....	14
Threads no Servidor.....	14
Threads no Cliente.....	15
Conclusão.....	16
Github:.....	17

Introdução - Garden in Overworld

O desenvolvimento de Sistemas Distribuídos carrega o desafio inerente de orquestrar múltiplos processos separados geograficamente, garantindo que eles se comuniquem, sincronizem e tolerem falhas de forma coesa. O projeto *Garden-in-Overworld*, uma simulação interativa de uma fazenda compartilhada, tem servido como um laboratório prático para a aplicação progressiva destes conceitos.

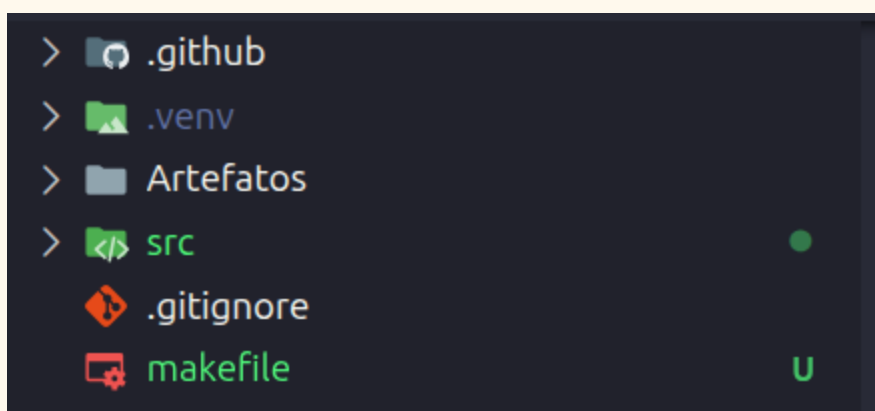
Em sua iteração anterior (Parte 3), o ecossistema do jogo foi fundamentado em uma arquitetura de Sockets TCP puros. Embora eficaz para o aprendizado dos fundamentos de rede, aquele modelo exigia um microgerenciamento severo: a aplicação precisava lidar manualmente com *threads* de escuta, empacotamento e conversão de *strings* em formato JSON, além do roteamento interno de cada requisição por meio de roteadores lógicos complexos.

Este relatório documenta a evolução e refatoração completa da arquitetura do sistema para a Parte 4, marcada pela adoção do *Middleware* RMI (*Remote Method Invocation*) Pyro5. O objetivo principal desta etapa é demonstrar, na prática, uma profunda mudança de paradigma: a transição de um modelo de troca de mensagens brutas para a elegância da transparência de rede. Ao delegar o peso da camada de transporte e serialização para o middleware, o foco do desenvolvimento retorna à Orientação a Objetos pura.

Ao longo das próximas seções, serão detalhadas as adaptações nas classes estruturais do cliente e do servidor, a nova topologia dependente de um *Name Server* (*Serviço de Nomes*), as estratégias adotadas para lidar com os limites da comunicação puramente síncrona do RMI (como o uso do padrão *Polling*) e, por fim, os impactos dessa abstração na qualidade, concorrência e manutenibilidade do código-fonte.

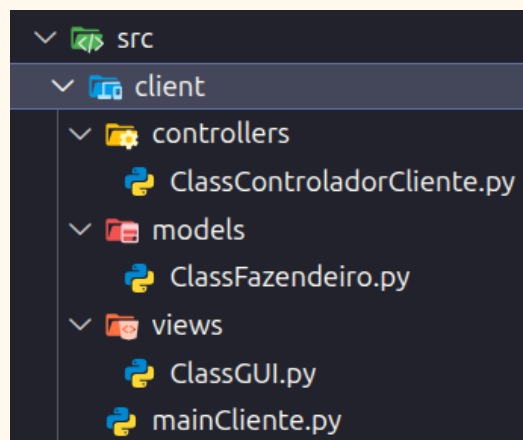
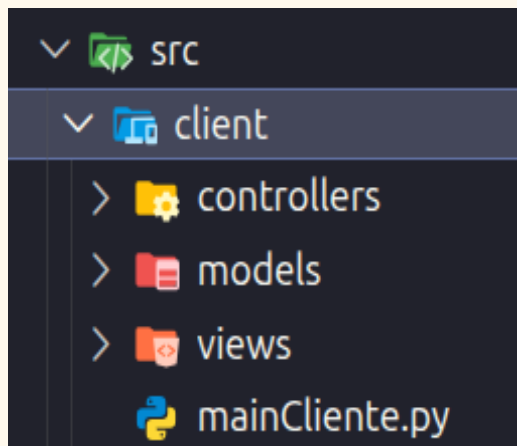
Organização:

Tanto o servidor, quanto o cliente foram projetados seguindo o padrão arquitetural de separação de responsabilidades (MVC/Injeção de Dependência). Visando isolar completamente a lógica de baixo nível da rede (Sockets e gerenciamento de buffers) das regras de negócio do jogo (validação de plantio, gerenciamento de mapa e inventários) e maneiras de visualização (frontend e terminal). Assim seguindo os padrões de qualidade de software e ciclo de vida aprendidos em Engenharia de Software.

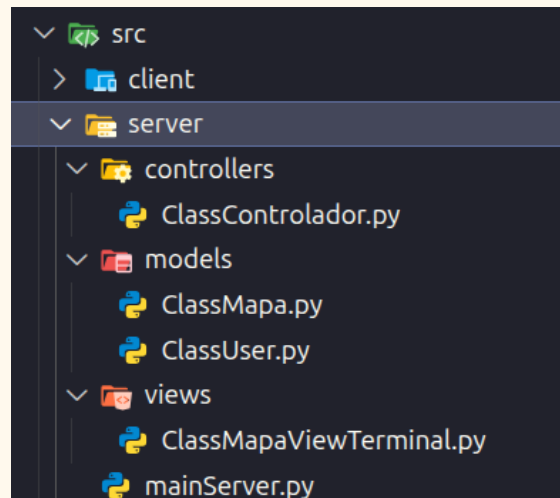
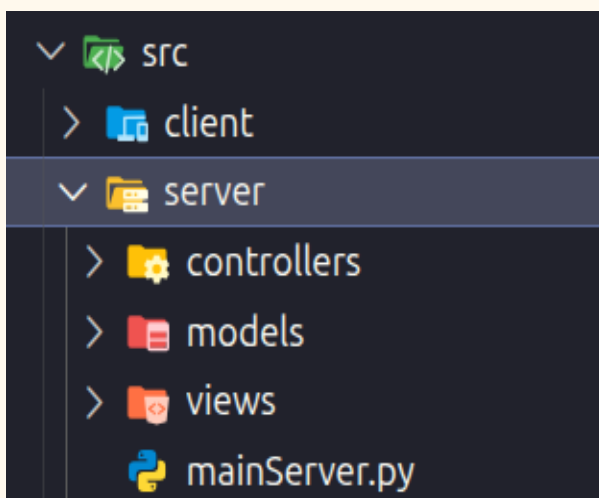


No repositório src/ encontram-se arquivos fonte para o servidor e a replicação de clientes.

Cliente:



Servidor:



Componentes (Servidor):

Camada Controllers

ClassControlador.py

- **Responsabilidade:** Atuar como o objeto remoto central do jogo disponibilizado pelo middleware RMI (Pyro5). Diferente da versão anterior, ele não atua mais como um roteador de strings ou conversor de JSON. Sua responsabilidade agora é expor de forma direta e limpa as regras de negócio da fazenda (conectar, plantar, colher) para que os clientes as invoquem como métodos nativos.
- **Funcionalidades e Permissões:** Funciona como a ponte orientada a objetos ligando o Middleware de Rede, o Mapa e o Gerenciador de Usuários. É a única classe do servidor com permissão de exposição na rede, utilizando o decorador `@Pyro5.api.expose` para permitir que suas funções internas sejam acessadas e executadas remotamente pelos processos clientes de forma transparente.
- **Escolha(s) de Implementação Crítica:** Instância Única e Abstração de Rede. Para garantir que todos os jogadores interajam simultaneamente com o mesmo estado do mundo, a classe é obrigatoriamente envelopada com o decorador `@Pyro5.api.behavior(instance_mode="single")`. A implementação abriu mão do antigo "Escudo Anti-Crash" manual, delegando o tratamento de anomalias de rede e serialização de dados para o próprio motor do Pyro5. Além disso, a comunicação foi refatorada para retornar estruturas de dados nativas do Python (como dicionários e a matriz do mapa) diretamente no final de cada função, substituindo o antigo sistema de broadcast ativo do servidor por respostas síncronas que alimentam a interface do cliente.

```
@Pyro5.api.expose
@Pyro5.api.behavior(instance_mode="single") # Garante que todos os jogadores usem a mesma instância da fazenda
class ControladorFazenda:
    def __init__(self, mapa_instancia, gerenciador_instancia):
        self.mapa = mapa_instancia
        self.gerenciador = gerenciador_instancia
```

Camada Models

ClassMapa.py

- **Responsabilidade:** Manter a matriz bidimensional representativa do mundo físico do jogo e aplicar as restrições geográficas de cultivo.
- **Funcionalidades e Permissões:** Controla de forma exclusiva as mutações dos blocos (Tiles). É responsável por gerar proceduralmente elementos como oásis (água cercada de areia), rios (verticais ou horizontais) e poças.

```
#aleatorizando mapa
num_oasis = random.randint(3, 6)
direcao = random.choice(["horizontal", "vertical"])
num_nascentes = random.randint(3, 6)
prob_areia = random.uniform(0.3, 0.5) # Probabilidade de cada célula ser areia
num_pocas = random.randint(3, 6)
maxtam_pocas = random.randint(2, 4)

mapa_jogo.gerar_mapa_aleatorio(num_oasis, direcao, num_nascentes,
                               prob_areia, num_pocas, maxtam_pocas)
mapaV.MapaView.exibir_colorido(mapa_jogo.matriz)
```

- **Regras de Negócio Implementadas:**
 - Trigo (ID 3): Só pode ser plantado em blocos de Terra (ID 0).
 - Arroz (ID 4): Só pode ser plantado exclusivamente em blocos de Água (ID 1).
 - Cana-de-Açúcar (ID 5): Pode ser plantada em Terra ou Areia, transformando-se dinamicamente em ID 6 (Cana na Terra) ou ID 7 (Cana na Areia), respectivamente. Essa transformação dinâmica serve pra conseguir voltar para o estado anterior antes de plantar sem alterar a composição original do mapa

ClassUser.py

- **Responsabilidade:** Gerenciar as credenciais voláteis (Nicks) dos jogadores, seus inventários individuais de colheita e o controle de vagas ativas no servidor.
- **Funcionalidades e Permissões:** Limita o acesso ao servidor a no máximo 4 jogadores simultâneos por meio de um mapeamento estático de slots (1 a 4). Realiza a alocação de nomes temporários (ex: "Jogador1") no momento da conexão, permitindo a posterior alteração por meio de requisições específicas.

Camada Views

`ClassMapViewTerminal.py`

- **Visão Geral:** Trata-se apenas da visualização por meios de terminal para vermos um pré-visualização do mapa gerado, no terminal do servidor, que será renderizado pelo lado do cliente.

`mainServer.py`

- **Responsabilidade:** Atuar como o Bootstrap (inicializador) do sistema distribuído. É responsável por instanciar o estado inicial do mundo (geração aleatória da matriz do mapa com oásis, rios e poças) e estabelecer a infraestrutura do Middleware RMI, conectando as regras de negócio locais à rede para que se tornem acessíveis aos clientes.
- **Funcionalidades e Permissões:** Possui a permissão de instanciar os modelos principais da aplicação (Mapa e Gerenciador de Usuários) e injetá-los no Controlador. Funciona como o único ponto de contato do lado do servidor com o Name Server do Pyro5. Ele inicializa o Daemon (processo em segundo plano que escuta as chamadas remotas) e registra o Controlador na rede sob um nome lógico e padronizado.
- **Escolha(s) de Implementação Crítica:** Delegação de Transporte e Autodiscovery. A escolha de arquitetura mais crítica nesta transição foi a completa remoção da manipulação manual de portas e sockets TCP/UDP. Toda a camada de transporte e alocação de conexões foi delegada para o Pyro5.

Componentes (Cliente)

Camada Controller

ClassControladorCliente.py

- **Responsabilidade:** Orquestrar a comunicação bidirecional entre a Interface Gráfica (View) e o Proxy RMI do servidor (Model). Traduz as interações do usuário (cliques na matriz) em chamadas de métodos remotos e aplica as respostas síncronas de volta na tela.
- **Funcionalidades e Permissões:** Possui permissão para manipular as propriedades visuais da janela do PyQt6 e acionar os métodos da classe ClienteFazenda. Diferente da versão baseada em Sockets, que utilizava uma Thread secundária (OuvinteThread) rodando em loop infinito, este controlador gerencia o tempo da aplicação por meio de um objeto QTimer nativo do framework visual.
- **Escolha(s) de Implementação Crítica:** (Transição de Push para Polling) .Na Parte 3 do trabalho (Sockets), o cliente adotava um modelo reativo (Push), onde a interface só era atualizada quando o servidor empurrava um broadcast. Devido à arquitetura puramente síncrona de requisição-resposta do Pyro5, a implementação de Callbacks (para simular o Push) traria extrema complexidade ao gerenciamento de threads da GUI. A escolha arquitetônica adotada foi a implementação do padrão Polling: um QTimer foi configurado para invocar o método solicita_matriz() a cada 1 segundo (1000ms), buscando ativamente o estado global do jogo e garantindo a sincronização em tempo real entre todos os jogadores com um custo computacional justificado pela estabilidade do cliente.

Camada Models

ClassFazendeiro.py

- **Responsabilidade:** Atuar como o representante local (Stub ou Proxy) do servidor remoto. Ele substitui completamente a antiga camada de Sockets TCP e a formatação manual de strings JSON, permitindo que o cliente interaja com o servidor como se este fosse um objeto residente na sua própria memória local.
- **Funcionalidades e Permissões:** Possui a permissão exclusiva de se comunicar com o Middleware Pyro5 no lado do cliente. Utiliza o método `Pyro5.api.locate_ns()` para localizar o "Nome Server" e o `Pyro5.api.Proxy()` para invocar os métodos da fazenda remota. Ele expõe uma API limpa para o Controlador, mascarando toda a complexidade do empacotamento dos dados que trafegam pela rede.
- **Escolha(s) de Implementação Crítica:** Transparência de Rede e Tolerância a Falhas. A implementação isola as chamadas do Pyro5 dentro de blocos `try/except`. Como o RMI executa operações síncronas que dependem da rede, qualquer queda do servidor ou do Name Server poderia causar o travamento ou o "crash" da interface gráfica do PyQt6. Ao isolar os erros nesta classe e retornar dicionários padronizados com o status "ERRO", garante-se que a camada visual continue responsiva e consiga exibir pop-ups de alerta amigáveis ao usuário caso a conexão seja perdida.

Camada View

ClassGUI.py

- **Responsabilidade:** Construir, dispor e atualizar os elementos tridimensionais, botões e campos visuais do usuário na tela através do framework PyQt6.
- **Isolamento Arquitetural:** Em conformidade com o padrão MVC, o arquivo ClassGUI.py é inteiramente agnóstico à rede. Ele não faz importações do módulo socket ou de JSONs e não sabe da existência de servidores. Classes gráficas como MatrixCell (que representam individualmente cada quadrado do mapa) e GameWindow expõem sinais de controle e métodos públicos de mutação gráfica (como atualizar_visual(valor)) para permitir que agentes externos governam seu comportamento estético.

mainClient.py

- **Responsabilidade:** Atuar como o Bootstrap do lado cliente. Inicia o loop de eventos da interface gráfica (QApplication), gerencia a tela de login inicial e constrói a árvore de dependências injetando o modelo de rede e a interface visual no Controlador.
- **Funcionalidades e Permissões:** Controla o ciclo de vida do processo do cliente no sistema operacional. Avalia imediatamente se o RMI está disponível e se há vagas na fazenda antes de permitir a renderização da janela principal (GameWindow).
- **Escolha(s) de Implementação Crítica:** Desconexão Graciosa via Hook de Sistema. Em sistemas baseados em Sockets TCP, a interrupção abrupta do processo (como fechar a janela no "X") quebra a conexão, permitindo ao servidor detectar a saída e liberar o slot do jogador. Em contrapartida, no Pyro5 (RMI), o servidor não mantém uma conexão ativa e contínua com o cliente, gerando o risco de "jogadores zumbis" ocuparem as vagas permanentemente caso a aplicação seja encerrada sem aviso. Para contornar essa limitação do middleware, foi implementado um Hook no evento nativo app.aboutToQuit. Essa escolha garante que, instantes antes do processo ser destruído pelo sistema operacional, uma última invocação remota (solicita_sair) seja disparada, notificando o servidor e mantendo a integridade da contagem de vagas.

Middleware VS. Sockets

A principal diferença na implementação encontra-se em como foi preciso tratar as trocas de mensagens entre as entidades arquitetônicas. Os middlewares são componentes que nos servem como meio de abstração, e assim foi, ao adicionarmos essa camada lógica, o tratamento de mensagens e o entendimento, assim como todo o fato de lidar com o baixo nível de trocas de mensagens remotas. Os sockets foram abstraídos por chamadas de funções que assemelha-se muito a chamadas locais, no qual o middleware responsabilizava-se pela padronização, confiabilidade e segurança dentro de seus limites.

O paralelismo (as threads), assim como as regras de negócio e o front end não precisaram ser alterados, já que não conversavam diretamente com a rede.

Alterações:

1. **Paradigma de troca de mensagens:** com a abstração feita pelo Pyro5, foi possível deixar o padrão json de lado, tirando a responsabilidade de compatibilidade na troca de mensagens de quem programava. Foi possível fazer chamadas de funções que retornassem apenas dicionários. Basicamente o Pyro5 cuida de decodificar os dados da rede automaticamente.
2. **Decoradores (@expose):** Foi usada a notação: `@Pyro5.api.expose` no topo da classe que serve para transformar todos os métodos públicos desta classe em funções que qualquer computador na rede pode executar. Fazendo assim, a classe em si virar o um “servidor”.
3. **Chamada Direta de Métodos:** O cliente podia enxergar o servidor como se fosse um objeto rodando no próprio computador dele. Assim foi possível executar no cliente chamando diretamente os métodos das classes presentes no servidor (apenas os métodos públicos).
4. **Broadcast vs. Polling:** Como as chamadas no RMI são puramente de requisição-resposta (síncronas), o modelo de broadcast ativo implementado via Sockets tornou-se inviável sem adicionar grande complexidade de callbacks. A solução adotada foi a implementação do padrão Polling. No cliente, em vez de manter uma classe ou thread exclusiva ouvindo a rede, foi utilizado um temporizador nativo da interface gráfica (QTimer) que realiza invocações contínuas (a cada 1 segundo) solicitando o

estado atualizado da matriz ao servidor, garantindo a sincronização em tempo real sem causar conflitos de concorrência na interface visual.

5. **Name Server e Desacoplamento:** Diferente da implementação com Sockets TCP, onde os clientes precisavam conhecer o endereço IP e a porta exatos do servidor de forma estática (hardcoded), o RMI introduziu o uso obrigatório do Name Server. O servidor agora registra seu objeto sob um nome lógico (como "fazenda.servidor"). Os clientes apenas localizam a "Lista Telefônica" na rede e pedem a referência por nome. Isso abstrai a topologia física da rede, permitindo que o servidor mude de IP ou porta sem que o código do cliente precise ser alterado.
6. **Controle de Instância:** Para garantir a consistência do mundo compartilhado do jogo, foi necessário utilizar o decorador `@Pyro5.api.behavior(instance_mode="single")`. Em Sockets, havia apenas um programa rodando em loop segurando as variáveis. No Pyro5, o comportamento padrão poderia criar um novo objeto controlador para cada cliente que se conectasse. O modo "single" assegura o padrão Singleton no middleware, garantindo que todos os jogadores interajam simultaneamente com a mesma instância da matriz e do gerenciador de usuários.
7. **Gerenciamento de Estado e Desconexões:** No modelo de Sockets, o sistema operacional avisa o servidor imediatamente quando a conexão de um cliente é interrompida abruptamente (como fechar a janela), permitindo limpar o slot do jogador instintivamente. No RMI, a comunicação é sem estado contínuo (stateless entre as chamadas). Para evitar que clientes desconectados ocupassem vagas permanentemente ("jogadores zumbis"), foi necessário adaptar a interface gráfica (via evento `app.aboutToQuit`) para realizar uma chamada remota explícita de `solicita_sair()` milissegundos antes do encerramento do processo cliente, devolvendo a responsabilidade de gerenciar a sessão para a camada da aplicação.

Dificuldades:

1. Por mais que o conceito geral de um middleware seja de extrema facilidade conceitual, ainda sim houve muita dificuldade em entender internamente o funcionamento dessa tecnologia/ camada lógica. Como se comportava, seus limites, às visualização de futuros problemas que pudessem causar e como realmente lidava com o SO e a camada de rede e transporte.
2. Não é bem uma dificuldade, mas é algo para se ter cuidado , já que aparentemente o modo de chamar as funções nos dá a impressão de estarem localizadas na mesma

entidade arquitetônica, o que pode nos dar a perda de noção de problemas comuns no universo de redes de computadores, como a latência e jitter.

Uso das Threads (Refatoração para Middleware RMI)

Para suportar múltiplos usuários jogando e interagindo simultaneamente no mesmo espaço geográfico, o controle de concorrência permanece essencial. Contudo, a migração para o middleware Pyro5 abstraiu grande parte do gerenciamento manual de linhas de execução que existia na implementação com Sockets TCP.

Threads no Servidor

Thread Principal (Daemon do Pyro5): Em vez de permanecer travada em um laço manual de `socket.accept()`, a thread principal agora é inteiramente delegada ao método `daemon.requestLoop()`. Sua única função é manter o serviço do middleware vivo, escutando a rede e interceptando as invocações remotas.

Gerenciamento Abstraído de Clientes : O middleware Pyro5 assume a responsabilidade de gerenciar as requisições concorrentes internamente (através de seu próprio pool de threads ou multiplexação). Quando múltiplos clientes invocam os métodos expostos da fazenda simultaneamente, o Pyro5 despacha essas chamadas automaticamente em paralelo.

Mecanismos de Sincronização e Concorrência (GIL): Mesmo com o Pyro5 roteando requisições simultâneas, o Global Interpreter Lock (GIL) do Python atua como a principal proteção do estado do jogo. Como as operações de leitura e gravação nas matrizes do mapa e nos dicionários de usuários ocorrem em blocos de bytecode intrinsecamente protegidos pelo GIL, o servidor consegue processar chamadas simultâneas (ex: requisições concorrentes da função `plantar()`) de forma atômica, eliminando o risco de corromper as estruturas de dados e a necessidade do uso de travas de exclusão mútua (Mutexes).

Threads no Cliente

Como regra padrão de interfaces gráficas, nenhuma operação bloqueante ou infinitamente demorada pode ser executada na Thread Principal, sob o risco de congelar a renderização da tela e tornar o programa inoperante para o sistema operacional. A arquitetura adotada na Parte 4 simplificou drasticamente este controle:

Remoção da Escuta em Segundo Plano (Retirada da OuvinteThread): Na versão anterior, o travamento inerente da função `socket.recv()` exigiu a criação da `OuvinteThread` (via `QThread`) e de um complexo barramento de sinais (`pyqtSignal`) para atualizar a tela sem violar as regras de concorrência gráfica e gerar falhas críticas (`Segmentation Faults`). Com a arquitetura síncrona de requisição e resposta do RMI, o cliente não "escuta" passivamente a rede, tornando a thread secundária completamente obsoleta.

Invocações Síncronas na UI: As requisições ativas (como o clique do usuário para plantar ou colher) são despachadas diretamente na Thread Principal pelo Controlador. Por lidarem com pacotes otimizados pelo middleware e resoluções locais de rede, a invocação remota e a recepção imediata do laudo (`{"validar": True/False}`) são processadas em frações de milissegundo, evitando qualquer congelamento (`overhead`) perceptível da interface (GUI).

Sincronização Passiva via Event Loop (Polling): Para consumir atualizações de estado causadas por outros jogadores sem o uso de threads extras, adotou-se o agendador nativo da interface gráfica: o `QTimer`. Ele executa chamadas periódicas (a cada 1 segundo) à função `solicita_matriz()` de forma assíncrona, diretamente dentro do Event Loop da Thread Principal. Essa escolha garantiu a sincronização em tempo real do mapa de forma fluida, limpa e nativamente Thread-Safe.

Conclusão

Os resultados obtidos com a refatoração e implementação desta etapa do projeto foram altamente satisfatórios. A transição da arquitetura de comunicação baseada em Sockets TCP para o middleware RMI (Pyro5) não apenas atendeu integralmente a todos os requisitos operacionais propostos para a simulação da fazenda *Garden-in-Overworld*, mas também demonstrou extrema estabilidade na sincronização de estado entre os múltiplos clientes. A nova topologia de rede, ancorada no *Name Server*, permitiu que as ações de plantio e colheita fossem validadas e replicadas visualmente sem apresentar travamentos na interface gráfica (GUI) ou condições de corrida no servidor, validando a eficácia do padrão de *polling* adotado.

Embora o conceito fundamental de um middleware seja de fácil assimilação teórica, compreender o seu funcionamento interno na prática revelou-se um desafio instigante. Entender como a tecnologia encapsula as camadas de rede e transporte, gerencia as chamadas junto ao Sistema Operacional e impõe seus próprios limites arquitetônicos exigiu uma análise profunda para prever e mitigar potenciais falhas sistêmicas.

Durante o desenvolvimento, observou-se um fator crítico de atenção inerente ao RMI: a extrema transparência das invocações. Como a sintaxe cria a forte ilusão de que os métodos executados pertencem à mesma entidade arquitetônica local, existe o risco de o desenvolvedor perder a sensibilidade para problemas clássicos e inevitáveis de redes de computadores, como variações de latência, perdas de pacotes e *jitter*. Contudo, essa falsa sensação de localidade foi rigorosamente observada e gerenciada no projeto através do isolamento de falhas no cliente e tratamento de exceções, garantindo a integridade do sistema.

Por fim, ao traçar um paralelo direto entre a Parte 3 (Sockets) e a Parte 4 (Middleware), a evolução do código-fonte é inegável. Na implementação baseada em Sockets TCP, havia um controle absoluto da infraestrutura de rede, o que permitia um modelo eficiente de notificações em tempo real (*Broadcast/Push*), mas exigia um microgerenciamento complexo de *Threads*, serialização manual de *strings* (JSON) e roteamento de mensagens. O middleware Pyro5 absorveu toda essa complexidade operacional, resultando em um código muito mais limpo, modular e focado puramente na Orientação a Objetos.

Em suma, a transição cumpriu o objetivo de simplificar o desenvolvimento distribuído, elevando o nível de maturidade do projeto.

Github:

Cada iteração ou versão do trabalho encontra-se em uma branch do diretório abaixo (os nomes são auto explicativos).

None

`git@github.com:deyrik/Garden-in-Overworld.git`

github.com/deyrik/Garden-in-Overworld/tree/feature/middleware

Referência(s):

Uso de IA: Gemini para tirar dúvidas e produzir a arte da capa da documentação. Nenhum código foi produzido por agentes.

<https://github.com/irmen/Pyro5/blob/master/docs/source/tutorials.rst>